



INSTITUTO POLITÉCNICO NACIONAL

Escuela Superior de Cómputo

ESCOM



Materia: Bases de Datos

Profesor: Gabriel Hurtado Aviles

Grupo: 3CV4

Título:

Práctica 1: "Modelo Entidad Relación"

Miembros de Equipo:

Gonzalez Carapia Brandon Uziel

Introducción

A lo largo de esta práctica se resolverán algunas preguntas así como poner en práctica lo visto hasta el momento en clases y poniendo en práctica lo que se llegue a investigar, con el objetivo de familiarizarse con un sistema como es el Git/Github realizando algunos ejercicios y aplicándolo en ciertos casos de nuestro interés.

Práctica 1: "Modelo Entidad Relación"

Objetivo: Que usted describa los sistemas de base de datos a partir de sus aspectos básicos, construya el entorno de trabajo que empleará durante todo el semestre —control de versiones y un sistema gestor de bases de datos ejecutado en un contenedor—, sustente su investigación con literatura científica arbitrada y aplique lo anterior al modelado conceptual de un problema real. Toda la evidencia se entrega en un repositorio de Git.

Esta práctica cubre la Unidad Temática I completa y, al mismo tiempo, establece la forma de trabajo del resto del curso. Se divide en cinco ejercicios: los dos primeros construyen el entorno técnico, el tercero y el cuarto sustentan la parte conceptual, y el quinto la aplica a un caso concreto. La entrega no es un archivo suelto: es un repositorio de Git donde quedará registrado cómo trabajó usted, no solo el resultado final.

Ejercicio 1. Control de versiones con Git y GitHub

Parte A. Investigación. Redacte al menos una cuartilla que responda lo siguiente:

1. Qué es un sistema de control de versiones y qué problema concreto resuelve en un trabajo en equipo.

Un sistema de control de versiones (VCS) es una herramienta de software diseñada para registrar los cambios realizados sobre un conjunto de archivos en un periodo de tiempo. Su propósito principal es permitir la recuperación de versiones específicas de un programa o proyecto, auditar las modificaciones efectuadas y facilitar la colaboración coordinada entre múltiples desarrolladores y equipos de trabajo.

2.Cuál es la diferencia entre Git y GitHub. No son sinónimos.

Git es un sistema de control de versiones distribuido, que ayuda en el mantenimiento de una gran cantidad de código, y que ofrece ciertos servicios a equipos compuestos por diversos programadores y GitHub funciona como un servidor en el que es posible almacenar repositorios de proyectos, agregando varias características para efectuar una gestión tanto de código como de un proyecto.

3. Defina, con sus palabras y con un ejemplo: repositorio, confirmación (commit), rama (branch), fusión (merge), conflicto de fusión, Pull Request, archivo .gitignore y archivo README.

- Repositorio: Es como una bóveda de datos que almacena los archivos de un proyecto junto con un historial de operaciones o cambios aplicados por algún miembro del equipo que esté trabajando. *Ejemplo: El directorio practical-bd/ que contiene todo el código y la carpeta oculta .git .*
- Confirmación(commit): Es como un cambio hecho por algún miembro en un determinado tiempo en el que guarda un “hasheo” con la información del cambio hecho. *Ejemplo: git commit -m "code:Cambio en notas de sintetizador" .*
- Rama(Branch):Es una rama de un árbol por así decirlo que apunta a un commit o cambio específico y que permite trabajar sobre esa sin alterar la rama principal llamada main o master. *Ejemplo: La rama Carapia creada para configurar el entorno del mapa y de los npc's.*
- Fusión(Merge):Función que combina un cambio de un commit a otro cambio,por ejemplo en mi proyecto cuando teníamos que poner o agregar funcionalidades ponemos el merge del cambio v3 al cambio main. *Ejemplo: Incorporar los cambios terminados de Carapia dentro de main .*
- Conflicto de fusión:Según yo era cuando nos daba el github el error de compatibilidad porque los cambios no se podían unir nos paso mucho cuando modificamos la misma función y nos dijeron que era porque pues el git no sabia cual agarrar como línea final por así decirlo.
- Pull Request:Es como la confirmación para integrar los cambios de una rama a otra. *Ejemplo: Un PR solicitando revisar el modelo ER antes de unirlo a la rama main .*
- Archivo .gitignore: Es el archivo de texto que especifica los patrones de búsqueda para indicar a git que archivos o directorios no deben de ser rastreados por la plataforma. *Ejemplo: El archivo *.log. del programa*
- Archivo README: Era como un manual de ejecución de un proyecto en el que integra lo necesario para usarlo,así como lo necesario para su instalación en un medio como VS. *Ejemplo: El archivo README.md que contiene la ficha técnica de la Práctica 1.*

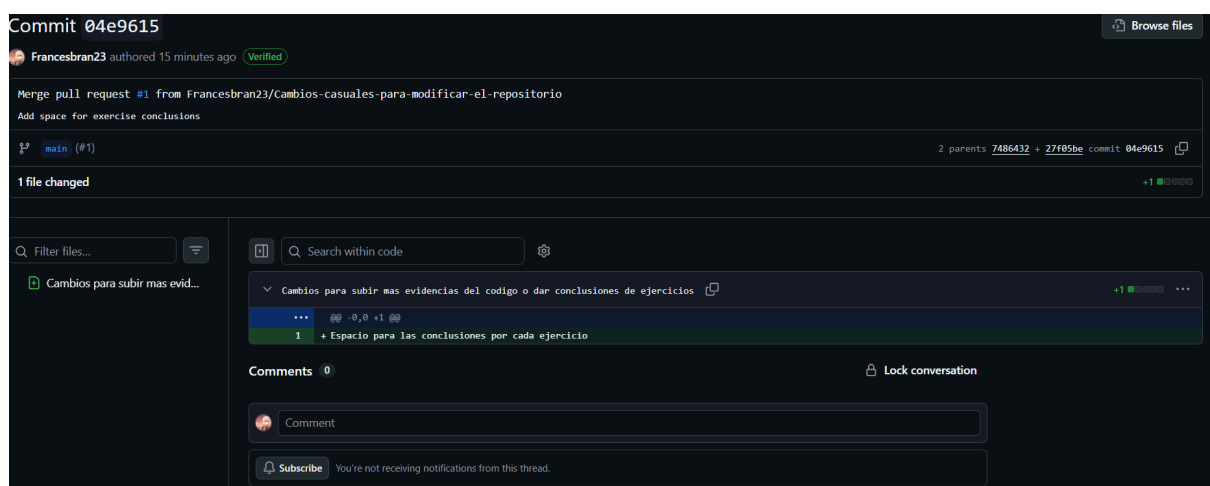
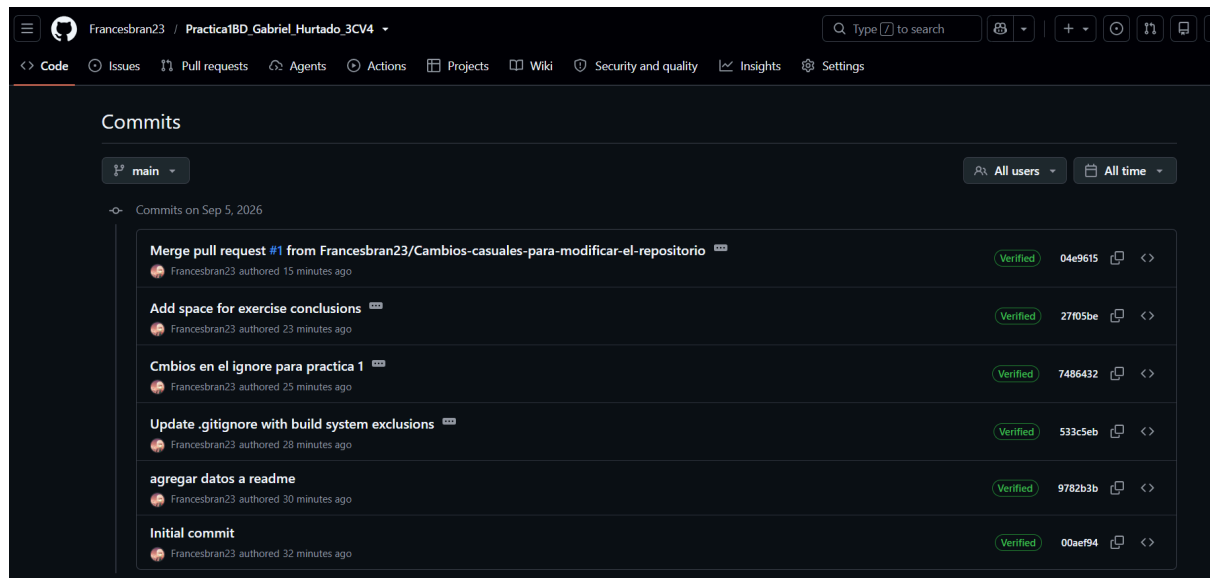
4. En qué consiste un flujo de trabajo basado en ramas y por qué el código se revisa entre pares antes de fusionarse.

Un Git Flow consiste en establecer una regla operacional de que la rama principal llamada main o master siempre debe de contener código estable y listo para producir y todos los cambios o experimentaciones deben de ser desarrollados en ramas secundarias.

La revisión entre códigos pares es una práctica esencial donde otros miembros del equipo examinan minuciosamente el código propuesto en el Pull Request antes de su fusión, por lo que se asegura la calidad técnica del producto previniendo así vulnerabilidades en el sistema.

Parte B. Práctica. Realice lo siguiente:

1. Cree una cuenta en GitHub, si aún no la tiene, y un repositorio para esta práctica.
 2. Agregue un archivo README.md con su nombre completo, grupo, carrera y un índice de la práctica.
 3. Agregue un archivo .gitignore adecuado al contenido del repositorio.
 4. Realice al menos cinco confirmaciones con mensajes descriptivos. Un mensaje como «cambios» no cuenta.
 5. Cree una rama, haga en ella un cambio, abra un Pull Request, descríballo y fusiónelo.
- Evidencia: la dirección del repositorio, una captura de la salida de git log --online --graph --all y una captura del Pull Request ya fusionado.



Ejercicio 2. El sistema gestor en un contenedor: Docker

Parte A. Investigación. En media cuartilla explique:

1. Qué es un contenedor y en qué se diferencia de una máquina virtual, en términos de arranque, tamaño y aislamiento.
2. Defina: imagen, contenedor, volumen y puerto publicado.
3. Por qué el volumen es indispensable y qué ocurre exactamente si no lo declara.

Un contenedor es una unidad ligera que empaqueta por así decir una aplicación junto con sus dependencias, ejecutándose en el mismo sistema operativo del host.

Sus diferencias a comparación de una VM son:

Arranque: Un contenedor inicia en segundos porque comparte el kernel del sistema operativo, mientras que una VM necesita arrancar un SO desde el inicio.

Tamaño: Los contenedores suelen ser mucho más pequeños por lo anteriormente mencionado.

Aislamiento: Las VM ofrecen un aislamiento total a nivel de hardware, mientras que los contenedores aíslan procesos y recursos pero comparten el mismo kernel.

En cuanto a las otras definiciones:

Volumen: Mecanismo de almacenamiento que permite guardar datos fuera del ciclo de vida del contenedor.

Puerto Publicado: Configuración que expone un puerto interno del contenedor hacia el host, permitiendo el acceso desde el exterior.

El volumen de un contenedor es indispensable ya que son efímeros; Es decir al detener o eliminar un contenedor, todos los datos internos se pierden. Si no se declara un volumen, cualquier información generada desaparece al reiniciar o eliminar el contenedor.

Parte B. Práctica. Levante un sistema gestor de bases de datos sin instalarlo en su sistema operativo:

1. Instale Docker Desktop, o bien Podman si prefiere una alternativa sin demonio privilegiado.
2. Levante el gestor de su elección en un contenedor. Se recomienda PostgreSQL por ser el que se usará en el curso.
3. Declare la configuración en un archivo `compose.yaml` y versiónelo en el repositorio. No basta con haber escrito el comando una vez en la terminal.

4. Conéctese con un cliente (psql, DBeaver o pgAdmin) y cree una base de datos vacía llamada practica1.

5. Demuestre la persistencia: detenga y elimine el contenedor, vuelva a levantarlo con el mismo volumen y compruebe que la base de datos sigue existiendo.

Comando mínimo de referencia:

```
docker run -d --name pg-practica1 \
-e POSTGRES_PASSWORD=practica1 \
-p 5432:5432 \
-v pgdata:/var/lib/postgresql/data \
postgres:16
```

Evidencia: el archivo compose.yaml versionado, una captura de la conexión al gestor y una captura de la prueba de persistencia.

Nota. Si su equipo no admite virtualización, documente por escrito el motivo, realice la instalación nativa del gestor y entregue la investigación de la Parte A completa.

	Francesbran23 feat: agregar compose.yaml para postgres	b93e68d · now	
	.gitignore	Cmbios en el ignore para practica 1	2 weeks ago
	Cambios para subir mas evidencias...	Add space for exercise conclusions	2 weeks ago
	README.md	Update index section in README.md	1 hour ago
	compose.yaml	feat: agregar compose.yaml para postgres	now

Containers

[Give feedback](#)

Container CPU usage

0.00% / 1200% (12 CPUs available)

Container memory usage

0B / 3.79GB

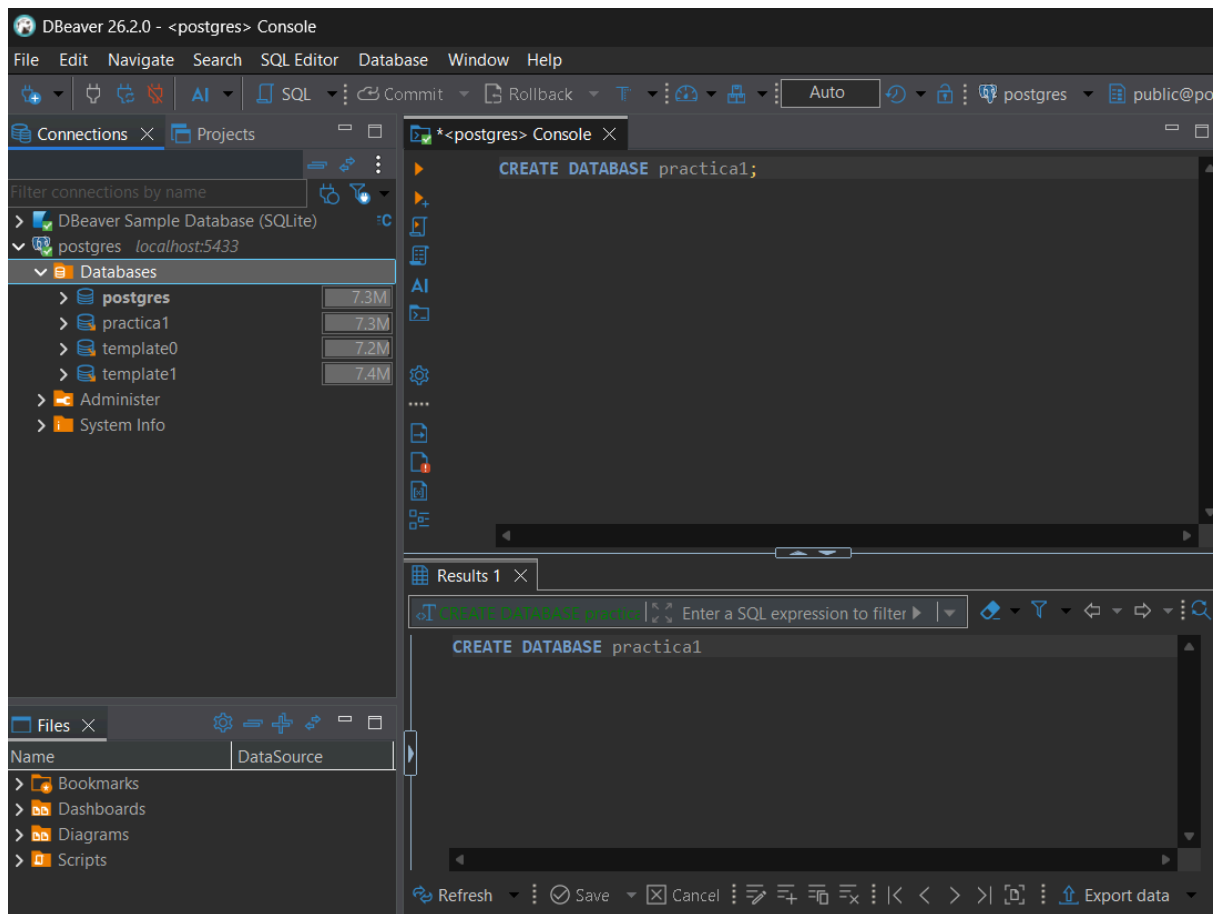
Show charts

Search

Only show running containers

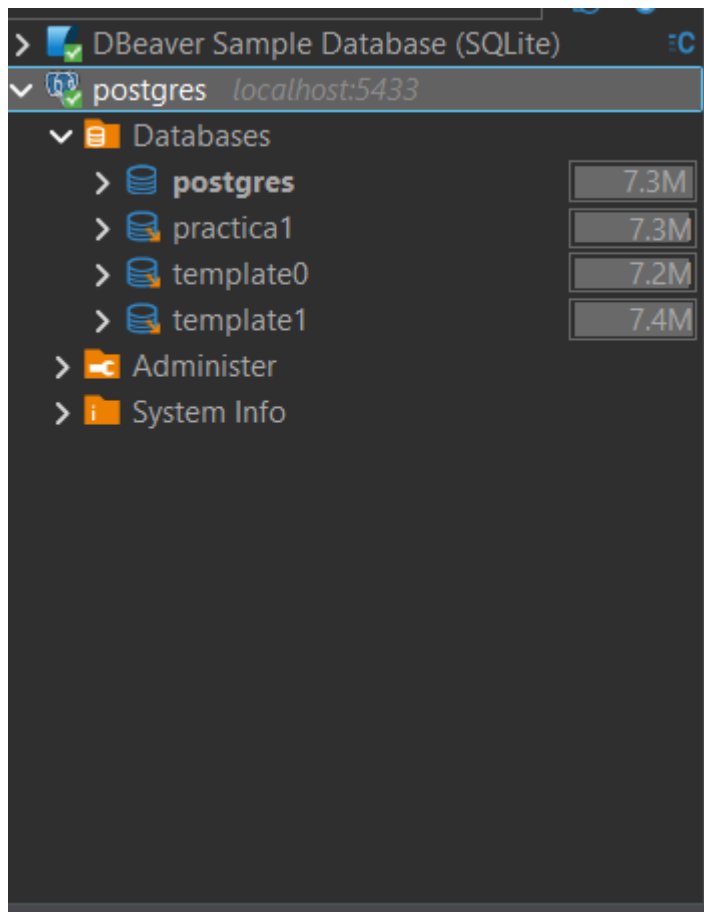
	Name	Container ID	Image	Port(s)	CPU (%)	Last	Actions
>	practica1bd_ga	-	-	-	0%	13 s	

```
PS C:\Users\gonza\OneDrive\Documentos\GitHub\Practica1BD_Gabriel_Hurtado_3CV4> git push
Enumerating objects: 4, done.
Counting objects: 100% (4/4), done.
Delta compression using up to 12 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (3/3), 463 bytes | 463.00 KiB/s, done.
Total 3 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
To https://github.com/Francesbran23/Practica1BD_Gabriel_Hurtado_3CV4.git
 21d9af8..b93e68d  main -> main
PS C:\Users\gonza\OneDrive\Documentos\GitHub\Practica1BD_Gabriel_Hurtado_3CV4> docker compose up -d
[+] up 20/20
✓ Image postgres:16                               Pulled
✓ Volume practica1bd_gabriel_hurtado_3cv4_pgdata    Created
✓ Network practica1bd_gabriel_hurtado_3cv4_default Created
✓ Container pg-practica1                            Started
PS C:\Users\gonza\OneDrive\Documentos\GitHub\Practica1BD_Gabriel_Hurtado_3CV4> █
```



```
PS C:\Users\gonza\OneDrive\Documentos\GitHub\Practica1BD_Gabriel_Hurtado_3CV4> docker compose down
[+] down 2/2
✓ Container pg-practica1 Removed
✓ Network practica1bd_gabriel_hurtado_3cv4_default Removed
PS C:\Users\gonza\OneDrive\Documentos\GitHub\Practica1BD_Gabriel_Hurtado_3CV4> ps -a
Get-Process : No se encuentra ningún parámetro que coincida con el nombre del parámetro 'a'.
En línea: 1 Carácter: 4
+ ps -a
+ ~~~
+ CategoryInfo          : InvalidArgument: (:) [Get-Process], ParameterBindingException
+ FullyQualifiedErrorId : NamedParameterNotFound,Microsoft.PowerShell.Commands.GetProcessCommand

PS C:\Users\gonza\OneDrive\Documentos\GitHub\Practica1BD_Gabriel_Hurtado_3CV4> docker compose up -d
[+] up 2/2
✓ Network practica1bd_gabriel_hurtado_3cv4_default Created
✓ Container pg-practica1 Started
PS C:\Users\gonza\OneDrive\Documentos\GitHub\Practica1BD_Gabriel_Hurtado_3CV4>
```



Conclusión

Gonzalez Carapia Brandon Uziel:

En esta práctica aprendí a implementar correctamente un prompt y script para la solución de compresión mediante la codificación de Huffman, utilizando una cola de prioridad basada en un heap de minimización construido desde cero. De igual forma, aunque un enfoque de compresión simple sería ineficiente, la estrategia de construir códigos de longitud variable según las frecuencias de los símbolos logra un ahorro significativo de bits, demostrando que a veces una reorganización del espacio de representación conduce a una compresión óptima sin pérdida de información. Comprendí que la clave está en emplear el heap de minimización, lo que permite acceder y extraer los nodos de menor frecuencia en tiempo logarítmico, garantizando que el algoritmo sea eficiente incluso para textos extensos.

Bibliografía

Git - About Version Control. (2026). Git-Scm.Com.

<https://git-scm.com/book/en/v2/Getting-Started-About-Version-Control>

PROGRAMACION WEB Full Stack 23 - Versionar el desarrollo: Git y GitHub. (2026).

Google Books.

https://books.google.com.mx/books?hl=es&lr=&id=0J9GDwAAQBAJ&oi=fnd&pg=PA2&dq=diferencias+entre+git+y+github&ots=7i6rIOuu_X&sig=R_BS7fTozsv3CwnBGufabWawvec&redir_esc=y#v=onepage&q=diferencias%20entre%20git%20y%20github&f=false